



University of Central Punjab

(Incorporated by Ordinance No. XXIV of 2002 promulgated by Government of the Punjab)

FACULTY OF INFORMATION TECHNOLOGY

Programming Fundamentals -LAB

INSTRUCTIONS:

- Midterm is self-explanatory, no query will be entertained.
 - Submit only **.c files**. (In case of multiple files, compressed into zip folder)
 - 50% deduction in case of run time error (**segmentation fault**).
-
- 70% deduction in case of **Logical error**.

CLO	CLO STATEMENT	Bloom's Taxonomy Level	PLO
1	Solve real-world problems skillfully with precision using programming constructs learned in theory with the course toolkit	P3	5

Project 1: Smart Hostel Management System

Project Summary:

Design and implement a complete **Smart Hostel Management System** for a university's residential hostel using C++. This system will help administrators to manage hostel students, assign rooms, track utility usage, generate bills, and maintain all records efficiently through a user-friendly console interface. Your system must use **functions** for modularity, **file handling** for data storage, **pointers** for dynamic data access, and **2D arrays** for representing rooms across blocks.

Hostel Room Allocation:

The hostel is divided into multiple blocks. Each block contains several rooms. Represent this using a **2D array**, where each row is a block and each column is a room. Each room has a room number, type (single/double), and occupancy status. When a student checks in, your program should allow the administrator to assign an available room of the desired type. A student must not be assigned more than one room, and a room can only be assigned to one student at a time.

Student Information:

Store each student's details including their **Student ID, Full Name, Gender, Age, Department, and Contact Number**. When a student checks in or out, their information should be updated in the system. Store this data permanently using **file handling**, so that the system retains the student list and room assignments between program executions.



University of Central Punjab

(Incorporated by Ordinance No. XXIV of 2002 promulgated by Government of the Punjab)

FACULTY OF INFORMATION TECHNOLOGY

Utility Billing Management:

Every room consumes electricity and water. At the end of each month, administrators enter the number of units consumed per room. The program should then calculate the bill based on fixed rates (e.g., Rs. 10/unit for electricity and Rs. 5/unit for water). Store and update these bills in files, and allow users to **view previous months' bills** or generate new bills. Let the user select a room and month to retrieve billing information.

Utility Billing Management:

Every room consumes electricity and water. At the end of each month, administrators enter the number of units consumed per room. The program should then calculate the bill based on fixed rates (e.g., Rs. 10/unit for electricity and Rs. 5/unit for water). Store and update these bills in files, and allow users to **view previous months' bills** or generate new bills. Let the user select a room and month to retrieve billing information.

Required Functionalities:

- Adding and storing student records
- Allocating and releasing rooms
- Viewing all available/occupied rooms
- Recording monthly utility usage
- Generating and viewing utility bills
- Searching a student by ID and showing their room and billing info

Make sure functions use **pass-by-reference and pointers** where appropriate — for example, when updating room status or reading student data from files.

Validations & Logic:

- Student ID must be numeric and unique.
- Room number must fall within defined range.
- Cannot allocate an already assigned room.
- Prevent billing with negative utility units.
- Validate contact number and student age.
- Use **nested if-else statements, loops, and file operations** for implementing all menu-driven functionalities.

File Structure Suggestions:

- students.txt → Store student records.
- rooms.txt → Store room availability and types.
- bills.txt → Store monthly bill summaries.
- All these should be opened and closed inside respective functions.



University of Central Punjab

(Incorporated by Ordinance No. XXIV of 2002 promulgated by Government of the Punjab)

FACULTY OF INFORMATION TECHNOLOGY

Sample Input:

Add New Student:

Student ID: 2023101

Name: Ali Raza

Age: 20

Gender: Male

Department: Computer Science

Contact: 03123456789

Allocate Room:

Block: 2

Room: 5

Room Type: Single

Utility Usage for June:

Electricity Units: 120

Water Units: 30

Sample Output:

Student added successfully and stored in file.

Room 2-5 allocated to Ali Raza.

Utility data recorded for Room 2-5 (June).

Bill Generated: Electricity Rs. 1200, Water Rs. 150, Total = Rs. 1350



University of Central Punjab

(Incorporated by Ordinance No. XXIV of 2002 promulgated by Government of the Punjab)

FACULTY OF INFORMATION TECHNOLOGY

Project 2: Vehicle Fleet Management and Trip Scheduling System

Project Summary:

Develop a **Vehicle Fleet Management and Trip Scheduling System** for a logistics company that owns and operates a large number of transport vehicles used for goods delivery across Pakistan. Your task is to design a C++ program that can manage all vehicles, assign them to delivery trips, track trip history, calculate fuel usage, and maintain trip logs. The system must use **functions** for modular design, **file handling** to store and retrieve all trip and vehicle data, **pointers** to dynamically access strings and structures, and **2D arrays** to track vehicle availability over the days of a month.

Vehicle Records Management:

Each vehicle in the fleet will have a **Vehicle ID, Type (Truck, Van, Container), Capacity in tons, Driver Name, and Availability Status**. You should allow the administrator to add new vehicles, update existing vehicle info, mark them as available/unavailable, and store these records permanently in a file (vehicles.txt). The system should prevent duplicate Vehicle IDs.

Trip Scheduling with 2D Availability Matrix:

To manage daily scheduling, use a **2D array** where rows represent vehicles and columns represent the days of a selected month (e.g., 1 to 30). Each cell should indicate whether the vehicle is scheduled for a trip on that day (T for trip, - for free). When a new trip is scheduled, the system must first verify if the selected vehicle is available on the requested day. If available, mark the vehicle as booked for that day. Each trip should have a **Trip ID, Source City, Destination City, Distance in KM, Date, Fuel Consumption (Liters), and Assigned Vehicle ID**. Trip records should be saved in a file named trips.txt.

Fuel Consumption & Billing:

For each trip, calculate the **fuel cost** by multiplying total liters consumed by the fuel rate (e.g., Rs. 275 per liter). The total cost for each trip should be shown along with a summary of trip details. Use functions to calculate fuel expenses, log trip info, and display trip history by Vehicle ID or Trip ID.

Required Functionalities:

Break your program into the following **well-named functions**:

- addVehicle(): Add a new vehicle and save it to file
 - scheduleTrip(): Assign vehicle to a trip after checking availability
 - viewFleetStatus(): Show 2D array of vehicle availability
 - calculateFuelCost(): Return total fuel cost of a trip
 - saveTripDetails(): Write trip data to file
 - viewTripHistory(): Load and display all past trips
- Use **pointers** when modifying vehicle or trip info inside functions.



University of Central Punjab

(Incorporated by Ordinance No. XXIV of 2002 promulgated by Government of the Punjab)

FACULTY OF INFORMATION TECHNOLOGY

Program Flow and Validations:

- Menu-driven interface for admin use
- Vehicle ID and Trip ID must be unique and alphanumeric
- Date format should be validated (e.g., 1 to 30)
- Fuel consumed and distance must be positive
- Prevent double-booking of vehicles on the same day
- Use **nested loops** to display the 2D trip calendar and search for trip slots.

File Structure Suggestions:

- vehicles.txt → List of all vehicles
- trips.txt → Records of scheduled trips
- calendar.txt → Store trip availability matrix

All file operations should use proper **error checking** to handle missing or corrupted files.

Sample Input:

Add New Vehicle:

Vehicle ID: V245

Type: Truck

Capacity: 8 Tons

Driver: Saeed Khan

Schedule Trip:

Trip ID: T102

Vehicle ID: V245

From: Lahore

To: Islamabad

Distance: 370 KM

Fuel Used: 80 Liters

Date: 15

Sample Output:

Vehicle V245 added successfully.

Vehicle V245 available on Day 15.

Trip T102 scheduled from Lahore to Islamabad.

Fuel Cost = 80 x Rs.275 = Rs. 22000

Trip details saved to file.



University of Central Punjab

(Incorporated by Ordinance No. XXIV of 2002 promulgated by Government of the Punjab)

FACULTY OF INFORMATION TECHNOLOGY

Project 3: University Examination Management and Grading System

Project Summary:

Build a complete **University Examination Management and Grading System** using C++ that simulates how a university department conducts exams, records marks, calculates grades, manages student records, and generates final result summaries. This program will make use of **functions** to structure the workflow, **file handling** for persistent record storage, **pointers** for efficient memory access to dynamic student data, and **2D arrays** to manage marks for multiple students and subjects.

Student & Subject Registration:

The administrator should first register all students who are appearing in the exam. For each student, store **Student Roll Number, Full Name, Program (e.g., BSCS, BSSE), Semester, and Section**. Use a **pointer array** to store student names dynamically, and make sure that each roll number is unique. Then, enter the list of subjects (e.g., Programming Fundamentals, Calculus, English) and store them as a 1D array of strings.

Exam Marks Entry Using 2D Arrays:

Allow the user to enter marks for each student in each subject. Use a **2D array** where each row represents a student and each column represents a subject. Marks must be validated between 0 and 100. The system should support up to **30 students and 6 subjects**. Input should be user-friendly and allow both batch entry and editing of specific student marks.

Grade Calculation and Result Sheet Generation:

Each student's percentage should be calculated based on the total marks obtained across all subjects. Then assign a grade using the following rules:

90–100% → A+

80–89% → A

70–79% → B

60–69% → C

50–59% → D

<50% → F

Grades should be stored in a separate 1D array. The system must also provide a **function to generate result cards** for each student showing detailed subject-wise marks, percentage, and grade.

File Saving and Searching:

Student marks, subject names, and final results should be saved in structured text files like marks.txt, students.txt, and results.txt. Use **file handling functions** to load existing records, update data, and append new entries. Also implement search functions to find a student by roll number and show their result card. Support should be included for reloading saved files when the program starts.

Required Functionalities Using Functions:

Use the following modular functions in your program:

addStudentDetails()

addSubjects()

enterMarks()

calculatePercentageAndGrade()

generateResultCard()



University of Central Punjab

(Incorporated by Ordinance No. XXIV of 2002 promulgated by Government of the Punjab)

FACULTY OF INFORMATION TECHNOLOGY

saveToFile() and loadFromFile()
searchStudentResult()

Make use of **pass-by-reference** and **pointers** to process student records and dynamically access mark arrays.

Program Flow and Validations:

- Allow maximum 30 students and 6 subjects
- Roll numbers must be unique
- Marks must be numeric and in valid range
- Provide option to re-enter or correct a student's marks
- Show overall class statistics like highest and average marks

Use **nested loops** for subject-wise mark entry, and **if-else-if** for grade classification. Make your interface menu-driven.

File Structure Suggestions:

- students.txt → Roll No, Name, Program, Semester
- marks.txt → Student-wise marks
- results.txt → Roll No, Percentage, Grade

These should be saved in plain text format and easily readable.

Sample Input:

Register Student:

Roll No: 23F-BSCS-101

Name: Areeba Khan

Program: BSCS

Semester: 1

Section: B

Enter Marks:

Subjects: Programming, English, Calculus

Marks: 90, 85, 75

Search Student Result:

Roll No: 23F-BSCS-101

Sample Output:

Student added successfully.

Marks entered and stored.

Result for Areeba Khan (23F-BSCS-101):

Programming: 90

English: 85

Calculus: 75

Total: 250/300

Percentage: 83.33%

Grade: A



University of Central Punjab

(Incorporated by Ordinance No. XXIV of 2002 promulgated by Government of the Punjab)

FACULTY OF INFORMATION TECHNOLOGY

Project 4: Airline Reservation and Ticketing System

Project Summary:

Develop a comprehensive **Airline Reservation and Ticketing System** for a local airline company operating multiple domestic and international flights. The system will allow passengers to book flights, check seat availability, manage passenger records, generate e-tickets, and maintain booking history. The program must use **functions** for logical organization, **file handling** to store all bookings and passenger data, **pointers** to manage string and structure operations, and **2D arrays** to simulate the seating plan of each flight.

Flight and Seating Management:

The airline operates 5 flights per day, each with a fixed seating arrangement of **10 rows and 6 seats per row**. Use a **2D array** to represent this seat matrix. Each seat can be marked as A for available and B for booked. When a passenger books a ticket, the system must show the seat layout, allow the user to choose a seat, and then mark it as booked. Ensure that no seat can be booked twice.

Each flight has a **Flight ID, Origin, Destination, Departure Time, and Date**. Save this flight information in a separate file named flights.txt. These flights must be preloaded at the start of the program.

Passenger Booking and Record Management:

Each booking will include **Passenger Name, CNIC, Contact Number, Flight ID, and Selected Seat Number (e.g., 5C)**. Passenger data should be stored permanently in a file called passengers.txt. The program should assign a unique **Ticket Number** to each booking and allow passengers to retrieve or cancel their ticket using their CNIC or ticket number.

Use **pointers** when accessing passenger structures and seat map data, especially during booking and cancellation.

E-Ticket Generation and Fare Calculation:

Each flight has a fixed **fare per seat**, which varies based on the route. For instance, a flight from Lahore to Karachi might cost Rs. 18,000 while Islamabad to Dubai could cost Rs. 45,000. When a passenger books a seat, generate an **e-ticket** showing all relevant details: Passenger Name, CNIC, Flight ID, Date, Seat Number, Fare, and Ticket Number. Store the e-ticket in tickets.txt.

Use **functions** to calculate fare based on origin/destination and generate ticket details. Tickets should be displayed on the screen and saved to the file system.

Required Functionalities Using Functions:

Your code must include the following modular functions:

- loadFlights() – Preload available flights into memory.
- showSeatMap() – Display 2D seat layout.
- bookSeat() – Assign seat and create ticket..
- cancelTicket() – Free seat and remove booking.
- generateETicket() – Print and save ticket.
- searchBooking() – Search by CNIC or ticket number.
- displayPassengerList() – Show all bookings for a flight.



University of Central Punjab

(Incorporated by Ordinance No. XXIV of 2002 promulgated by Government of the Punjab)

FACULTY OF INFORMATION TECHNOLOGY

Program Flow and Validations:

CNIC must be 13 digits, numeric only

No double bookings for the same seat

Passenger cannot book more than 1 ticket for the same flight

Validate seat input like "2F" properly (row and column)

Ticket number should auto-increment and be unique

The system should offer a **menu-driven interface** with options for booking, canceling, viewing flights, searching, and displaying reports.

File Structure Suggestions:

flights.txt → Static flight info (read-only)

passengers.txt → Passenger and booking info

tickets.txt → Complete ticket records

seats_F123.txt → Seat map file per flight (optional)

Sample Input:

Book New Ticket:

Passenger Name: Zainab Fatima

CNIC: 3520212345678

Flight ID: F123

Seat Number: 4B

Contact: 03331234567

Search Booking:

Ticket No: T1005

Sample Output:

Seat 4B is available on Flight F123.

Booking Confirmed!

Ticket Number: T1005

Passenger: Zainab Fatima

CNIC: 3520212345678

Flight: F123 | From: Lahore | To: Karachi

Departure: 9:30 AM | Date: 05-06-2025

Seat: 4B | Fare: Rs. 18000

Ticket saved successfully.



University of Central Punjab

(Incorporated by Ordinance No. XXIV of 2002 promulgated by Government of the Punjab)

FACULTY OF INFORMATION TECHNOLOGY

Project 5: Smart Grocery Store Inventory and Billing System

Project Summary:

Design a fully functional **Smart Grocery Store Inventory and Billing System** in C++ that allows the store owner to manage daily inventory, add or update product information, handle customer purchases, generate digital bills, apply discounts, and maintain transaction records. The program should incorporate **functions** for modularity, **file handling** for persistent data storage, **pointers** to dynamically handle product and billing data, and **2D arrays** to manage product categories and quantities.

Product Catalogue and Inventory Handling:

Begin by allowing the store administrator to add grocery products to the system. Each product entry should include **Product ID, Name, Category (e.g., Fruits, Beverages, Snacks), Unit Price, and Quantity in Stock**. Categories should be handled using a **2D array**, where each row is a category and columns store products under that category. All inventory data should be stored and updated in a file named `inventory.txt`. When a new product is added, it should be placed in its appropriate category in the 2D array and also written to the file. If a product with the same ID already exists, the system should prompt the user to update the quantity or price instead of duplicating it.

Customer Purchase and Cart System:

When a customer makes a purchase, they should be able to view all available products by category, select multiple products, specify quantities, and add them to their **cart**. The cart can be handled using an array of structures containing **Product ID, Name, Quantity Purchased, Unit Price, and Total Price for each item**. Pointers should be used to manage memory allocation of the cart dynamically.

After the cart is finalized, calculate the **total bill**, apply any relevant **discounts** (e.g., 10% discount if total bill exceeds Rs. 10,000), and generate a detailed receipt. All billing data must be stored in `bills.txt`.

Billing and Discount System:

Each bill should include the **Customer Name, CNIC, Cart Items, Gross Total, Discount (if any), and Net Total**. Include a billing ID that auto-increments. The receipt should be saved both on-screen and in a text file.

Create **functions** for:

- Adding products to inventory
- Updating product info
- Displaying product catalogue by category
- Adding products to the customer cart
- Calculating bill and discount
- Generating and saving bill
- Searching bills by CNIC or Bill ID

All operations must include **validations** to prevent negative pricing, over-purchasing stock, or invalid entries.

Product and Billing Files:

Use structured file handling:

- `inventory.txt` → Holds all current product records
- `bills.txt` → Holds each billing transaction line by line
- `sales_report.txt` (optional) → Summarizes daily or monthly sales

You may use append mode when adding new sales and overwrite mode for updating inventory levels after a



University of Central Punjab

(Incorporated by Ordinance No. XXIV of 2002 promulgated by Government of the Punjab)

FACULTY OF INFORMATION TECHNOLOGY

sale.

Required Functionalities Using Functions:

Design the following functions with clean separation:

```
addProductToInventory()  
updateInventory()  
displayProductsByCategory()  
createCustomerCart()  
calculateTotalBill()  
applyDiscount()  
generateReceipt()  
saveTransaction()  
searchBillByCNIC()
```

Use **pointers to arrays and structures** for cart operations and billing functions. Use **pass-by-reference** where necessary to update product quantities.

Program Flow and Validations:

- Ensure Product IDs are unique
- Quantity purchased must not exceed available stock
- Discount only applies if total exceeds threshold
- Cart must contain at least one item
- All user inputs must be validated (e.g., no negative prices or stock)

Provide a **menu-driven interface** with options for admin (add/update products) and customer (shop/view bill/search bill).

Sample Input:

Add Product:
Product ID: P101
Name: Nestle Juice
Category: Beverages
Unit Price: 180
Quantity in Stock: 50
Customer Shopping:
Name: Ali Raza
CNIC: 3520212345678
Add to Cart: P101, Quantity: 3

Sample Output:

Cart Updated: Nestle Juice x 3 @ 180 each = Rs. 540
Gross Total: Rs. 540
Discount: Rs. 0
Net Total: Rs. 540
Generating Receipt...
Bill ID: B1004
Customer: Ali Raza | CNIC: 3520212345678
Items Purchased:
- Nestle Juice x3 = Rs. 540
Thank you for shopping!



University of Central Punjab

(Incorporated by Ordinance No. XXIV of 2002 promulgated by Government of the Punjab)

FACULTY OF INFORMATION TECHNOLOGY

Project 6: Library Automation and Book Lending System

Project Summary:

Develop a robust **Library Automation and Book Lending System** in C++ that allows the librarian to manage books, track available and issued items, handle student registrations, generate book lending records, and maintain complete borrowing histories. The system should use **functions** to separate tasks, **file handling** to store student and book records, **2D arrays** to track book copies and genres, and **pointers** to manage dynamic user interactions and records.

Book Inventory and Classification:

The system should begin by allowing the librarian to input new books into the library inventory. Each book will have **Book ID, Title, Author, Genre, Edition, Total Copies, and Available Copies**. Use a **2D array** to group books by genre (e.g., Fiction, Non-fiction, Science, History, Biography), with a maximum of 20 books per genre. Each row in the 2D array represents a genre, and columns represent books under that genre. Ensure all book data is saved in a file named books.txt. When books are borrowed or returned, their available copy count must be updated both in memory and in the file.

Student Registration and Borrowing Rules:

Register students by taking inputs: **Roll Number, Name, Class, Section, and CNIC**. Each student is allowed to borrow up to **3 books at a time**. Use an array of structures to manage student borrowing records. Each record should include the **Book ID, Borrow Date, and Return Date**. Store student details in students.txt and borrowing activity in borrowings.txt.

You must use **pointers** to access and manage student borrowing records and available book copies during transactions.

Book Lending and Return Process:

When a student requests to borrow a book:

- Display all available books by genre.
- Allow the student to select a book by Book ID.
- Verify the student hasn't exceeded their borrowing limit.
- Update the book's available copies.
- Generate a **Borrow Receipt** showing the due date (14 days from issue).

When a book is returned:

- Ask the student to input their Roll Number and Book ID.
- Validate whether the book was actually borrowed.
- Update the inventory and borrowing history.
- Calculate any fine (e.g., Rs. 20/day after 14 days) and display the receipt.

Reports and Searching:

Add the ability to search books by title, author, or genre. Also, search students by Roll Number and show which books they currently have. Provide a **report generation** feature that lists all overdue books and the total fine amount for each student.

Required Functionalities Using Functions:

Break your program into the following functions:

- addBookToInventory()
- updateBookStock()
- registerStudent()
- borrowBook()
- returnBook()



University of Central Punjab

(Incorporated by Ordinance No. XXIV of 2002 promulgated by Government of the Punjab)

FACULTY OF INFORMATION TECHNOLOGY

searchBooks()
generateBorrowReceipt()
calculateFine()
generateReports()

Use **pointers** to structures when updating books and student borrowings. Use **file I/O** to maintain a persistent and recoverable record of all library activities.

Program Flow and Validations:

- Prevent borrowing if book is out of stock
- Prevent borrowing more than 3 books per student
- Validate return date and calculate fines accurately
- Book IDs and student Roll Numbers must be unique
- Borrowing history must be updated every time

Use **nested loops** for genre-wise book listing and fine calculation for overdue books. Use **if-else-if conditions** to determine book availability and validate borrowing rules.

File Suggestions:

- books.txt → Book records
- students.txt → Student information
- borrowings.txt → Active borrowings
- history.txt → Complete lending history

All files must be properly handled with open, close, append, and overwrite modes as needed.

Sample Input:

Add Book:
Book ID: B102
Title: Atomic Habits
Author: James Clear
Genre: Self-Help
Total Copies: 5

Register Student:
Roll No: 23F-CS-045
Name: Hammad Iqbal
Class: BSCS-1A
CNIC: 3520212345678

Borrow Book:
Roll No: 23F-CS-045
Book ID: B102

Sample output:

Book 'Atomic Habits' is available. 5 copies in stock.
Student Hammad Iqbal verified.
Borrowing record added. Due Date: 14-June-2025
Receipt:
Book: Atomic Habits | Borrow Date: 31-May-2025
Due in: 14 days
Thank you! Return the book on time to avoid fine.



University of Central Punjab

(Incorporated by Ordinance No. XXIV of 2002 promulgated by Government of the Punjab)

FACULTY OF INFORMATION TECHNOLOGY

Project 7: Vehicle Parking and Fine Management System

Project Summary:

Build a detailed **Vehicle Parking and Fine Management System** for a large commercial parking plaza. The system should allow the registration of incoming vehicles, assign parking spots, manage time-based billing, handle illegal parking fines, and maintain permanent parking records. The program must utilize **functions** for logical division, **file handling** for record storage, **2D arrays** to simulate the parking grid, and **pointers** for dynamic data referencing.

Parking Lot Structure and Vehicle Categories:

The parking plaza consists of **3 floors**, each containing **10 rows and 5 columns of parking spots**. Use a **2D array** for each floor to represent vehicle placements. Each spot can be empty (E), occupied by a car (C), bike (B), or van (V). Track floors using a 3D array-like approach with three 2D arrays. Ensure each vehicle is assigned to a valid and available slot. You must display the parking map to users during registration.

Each vehicle should be registered with:

Vehicle Number Plate

Type (Car, Bike, Van)

Owner Name

Contact Number

Entry Time and Date

This data should be stored in vehicles.txt. On exit, calculate the total time parked and generate a bill.

Billing and Fine System:

Each vehicle type has a specific per-hour rate:

Car → Rs. 100/hour

Bike → Rs. 50/hour

Van → Rs. 150/hour

If a vehicle parks illegally (e.g., without registration or wrong zone), apply a **flat fine of Rs. 5000**. Store fine records in fines.txt with details like plate number, offense type, and fine amount.

The system should:

- Calculate the total time parked (difference between entry and exit time)
- Apply hourly rates accordingly
- Detect if a vehicle overstays or parks in an unassigned slot
- Update the 2D array map when a vehicle leaves

Use **functions** to calculate bills, assign parking, validate zones, and manage fine entries.

Core Functionalities Using Functions:

Implement the following modular functions:

assignParkingSlot()

displayParkingMap()

registerVehicleEntry()

calculateParkingDuration()

calculateCharges()

vehicleExit()

generateBill()

applyFine()

searchVehicleByPlate()

generateDailyReport()



University of Central Punjab

(Incorporated by Ordinance No. XXIV of 2002 promulgated by Government of the Punjab)

FACULTY OF INFORMATION TECHNOLOGY

Use **pointers** to pass vehicle records and reference parking slots in arrays. Make heavy use of **nested loops** to manage and search the 2D parking arrays.

Program Flow and Validations:

- Do not allow more than one vehicle in a single slot
- Ensure no duplicate vehicle number registrations
- Check valid format for plate number (e.g., LHR-1234)
- Reject unrecognized vehicle types
- Apply fine immediately for unauthorized parking

The interface must be **menu-driven**, offering options for new entries, vehicle exit, fine reporting, slot viewing, and searching.

File Storage System:

- vehicles.txt → All registered entries
- bills.txt → All payment receipts
- finances.txt → Fines issued
- parking_map.txt → Parking grid saved state
- report.txt → Daily summary (optional)

Files should be opened in appropriate modes and closed properly. Ensure updates are persistent between runs.

Sample Input:

Register Vehicle:

Plate No: LHR-8754

Type: Car

Owner: Fatima Zahra

Contact: 03001234567

Entry Time: 10:30 AM

Date: 30-05-2025

Exit Vehicle:

Plate No: LHR-8754

Exit Time: 2:45 PM

Sample output:

Car assigned to Floor 2, Spot [5][2]

Time Parked: 4 hours 15 minutes

Hourly Rate: Rs. 100

Total Charges: Rs. 400

Generating Bill...

Plate: LHR-8754

Owner: Fatima Zahra

Entry: 10:30 AM | Exit: 2:45 PM

Total Bill: Rs. 400

Thank you for visiting!



University of Central Punjab

(Incorporated by Ordinance No. XXIV of 2002 promulgated by Government of the Punjab)

FACULTY OF INFORMATION TECHNOLOGY

Project 8: Wildlife Rescue Center – Animal Admission, Treatment, and Release Tracking System

Project Overview:

Design a detailed **Wildlife Rescue and Rehabilitation System** that models the operations of a real-life animal rescue centre. This C++ project will allow wildlife staff to **register injured or rescued animals**, **track their medical treatment progress**, and finally mark them as **ready for release**. The system will also maintain **daily logs**, **generate animal status reports**, and track rescue centre occupancy. You must implement this using **functions**, **file handling**, **2D arrays**, and **pointers**, and ensure the total project reaches approximately **300 to 400 lines of code**.

Animal Rescue Registration:

Your program should allow rescuers to register new animals brought into the centre. Each animal record must include:

- Animal ID (auto-generated, e.g., A-2025-001)
- Animal Type (e.g., Hawk, Fox, Turtle, Owl, Deer)
- Rescue Date
- Location Found
- Injury Description
- Initial Health Score (0 to 100)
- Assigned Vet Name

All entries must be saved in `rescue_animals.txt`. A pointer to a structure should manage individual animal data. Store up to **5 animal types**, each in their own **2D array** (5 types × 10 capacity each), representing treatment status across different zones in the rescue centre.

Medical Treatment Tracking:

Each day, a vet can update the animal's **health progress score**. Students must:

- Use **functions** to update health status.
- Maintain a file `treatment_log.txt` with the date, animal ID, vitals, and remarks.
- If a health score reaches 90+, the animal is marked **“Ready for Release”** and logged in `release_log.txt`.

You'll use **file handling** for persistent logs and apply **nested loops** to manage progress tracking. Each update must include:

- Feeding
- Medication
- Therapy
- Health Score Update

Release Scheduling and Habitat Match:

Once an animal is marked as "Ready for Release," allow the user to:

- Select a habitat region (e.g., Forest, River, Hills, Desert)
- Confirm environmental match
- Schedule the release date
- Mark animal status as "Released"

This must be handled with a pointer to the animal's structure, updating both memory and the `release_log.txt` file. Prevent early releases (before 90 health score) and keep daily release counts under 3.



University of Central Punjab

(Incorporated by Ordinance No. XXIV of 2002 promulgated by Government of the Punjab)

FACULTY OF INFORMATION TECHNOLOGY

Daily and Monthly Reports:

Create **functions** for:

- Viewing total animals currently in rescue

- Number of animals treated and released

- Count per species type

- Generate **monthly activity reports** stored in report_monthly.txt

Use **pointers** to process dynamic data lists and **file I/O** to read and summarize logs.

Required Functional Features:

Implement the following with **functions and pointers**:

- registerAnimal()

- updateTreatment()

- checkHealthProgress()

- scheduleRelease()

- generateReports()

- loadAnimalsFromFile()

- saveAnimalStatus()

Use **2D arrays** to represent:

- Rescue centre compartments.

- Daily treatment checklist per animal.

Validations:

- No duplicate Animal IDs

- Health score capped between 0–100

- Cannot release animal before meeting health threshold

- Only 3 animals may be released per day

- Require matching habitat for release

Sample Inputs:

Register Animal:

Type: Owl

Location Found: Dadu Forest

Rescue Date: 30-May-2025

Initial Health Score: 45

Assigned Vet: Dr. Saba

Daily Treatment Update:

Animal ID: A-2025-001

Feeding: Done

Medication: Antibiotic given

Therapy: Wing Movement Restored

Updated Health Score: 95

Schedule Release:

Habitat: Forest

Release Date: 04-June-2025



University of Central Punjab

(Incorporated by Ordinance No. XXIV of 2002 promulgated by Government of the Punjab)

FACULTY OF INFORMATION TECHNOLOGY

Sample Outputs:

Animal Registered with ID: A-2025-001

Treatment Log Updated:

Health Score: 95

Status: Ready for Release

Animal Released Successfully!

Habitat: Forest

Release Date: 04-June-2025

Report:

Total Animals Rescued: 12

Total Released: 3

Remaining in Center: 9

Project 9: TechFest Management and Participant Record System

Project Summary:

Imagine a university organizing a **TechFest** with multiple competitions, workshops, and guest speaker sessions. Develop a full-fledged **TechFest Management System** in C++ to handle event creation, participant registrations, score tracking, and winner announcements. This project must be implemented using **functions, pointers, file handling, and 2D arrays**, and must be written in a modular way requiring around **300 to 400 lines of code**. It should simulate a real-life event management system used in university tech festivals.

Core Functional Areas:

The system will be used by both:

Organizers (to add events, record scores, and generate reports)

Participants (to register, view schedules, and see results)

There will be at least **5 events** (e.g., Coding Competition, Quiz Show, Robotics Challenge, Gaming Arena, and Poster Design). Each event can hold a maximum of **30 participants**.

Event and Participant Registration:

Organizers should be able to:

Add new events with name, type, date, and time

Register participants to specific events

Check for time clashes and avoid double booking for any participant

Store all event and participant data in events.txt and participants.txt

Each participant's data includes:

Name

University Roll Number (must be unique)

Department

Contact

Selected Event



University of Central Punjab

(Incorporated by Ordinance No. XXIV of 2002 promulgated by Government of the Punjab)

FACULTY OF INFORMATION TECHNOLOGY

Time Slot

Participation ID (auto-generated, like TECHFEST-2025-017)

Use **pointers to structures** for participant info and event assignments.

Score Submission and Winner Declaration:

After an event finishes, organizers will:

Enter participant scores (out of 100) using **2D arrays**, one row per event, columns per participant

Use a function to find the **top 3 scorers**

Announce winners and update results.txt

Use file handling to store:

Event-wise scores

Top scorers

Participation logs

Ensure scores are validated and properly indexed by event and roll number.

Time Slot Management:

Prevent students from enrolling in two events scheduled at the **same time slot**. Implement a **time conflict checker** using a search function and a 2D array for event schedule. Store slot availability in memory and update as participants register.

If a time clash is detected:

Error: Participant already registered for another event at this time slot.

Reports and Features Using Functions:

Implement the following features through **modular functions**:

addEvent()

registerParticipant()

checkTimeClash()

submitScores()

findTopScorers()

generateCertificates()

viewEventSchedule()

generateReports()

Use **pointers** and **file I/O** in all data transfer and updates.

File Management Details:

Use the following files:

events.txt – All events and their details

participants.txt – All participant records

scores.txt – Event-wise score entries

results.txt – Top 3 winners of each event

certificates.txt – Winner name + event + position

All files should be updated automatically when scores are entered or events are closed.



University of Central Punjab

(Incorporated by Ordinance No. XXIV of 2002 promulgated by Government of the Punjab)

FACULTY OF INFORMATION TECHNOLOGY

Validation Rules:

No duplicate Roll Numbers or Participation IDs

Max 30 participants per event

No time clashes allowed

Scores must be between 0–100

Only registered students can view results

Sample Input:

Add Event:

Name: Coding Competition

Type: Technical

Date: 05-06-2025

Time Slot: 10:00 AM

Register Participant:

Name: Areeba Ahmed

Roll Number: 21-BCS-012

Department: CS

Contact: 0302-1234567

Event: Coding Competition

Time Slot: 10:00 AM

Enter Scores:

Event: Coding Competition

Participant: 21-BCS-012 → Score: 95

Participant: 21-BCS-045 → Score: 88

Participant: 21-BCS-031 → Score: 78

Sample Output:

Event Added Successfully!

Participant Registered: TECHFEST-2025-017

Scores Submitted for 'Coding Competition'

Top 3 Winners:

1st: Areeba Ahmed – 95

2nd: Bilal Khan – 88

3rd: Zara Malik – 78

Certificates Generated.



University of Central Punjab

(Incorporated by Ordinance No. XXIV of 2002 promulgated by Government of the Punjab)

FACULTY OF INFORMATION TECHNOLOGY

Project 10: Online Cinema Ticket Booking and Seat Reservation System

Project Summary:

Develop a **Cinema Ticket Booking System** that simulates the core operations of an online movie theater booking platform. This system will allow cinema staff to **add movies**, **create seating layouts**, **manage ticket sales**, and let users **book seats**, **view availability**, and **cancel bookings**. All major data (movies, seat maps, bookings, cancellations, revenue) must be stored using **file handling**, and the entire program should be structured modularly using **functions**, **pointers**, and **2D arrays** to represent cinema halls. This project must require a minimum of **300 to 400 lines of code**, and avoid overlapping with previous ideas such as hotel booking or transport systems.

Cinema Structure:

The cinema has **3 screens**, and each screen runs **one movie per day**. Each screen has a **seating layout of 10 rows and 10 columns**, represented as a **2D character array**, where:

'A' = Available seat

'B' = Booked seat

'X' = Blocked seat (for maintenance or distancing)

Each screen is associated with:

A Movie Name

Genre

Duration

Show Time

Ticket Price

This data is stored in a file named movies.txt.

Admin Functionalities:

Allow cinema staff (admin) to:

Add a new movie to a screen

Configure show time and pricing

Initialize the seating layout (default: all 'A')

Block certain seats for maintenance ('X')

View total earnings from bookings

This section should use **functions** like:

addMovie()

initializeSeating()

blockSeats()

viewRevenue()

All seat maps should be saved to and loaded from files like screen1_seats.txt, screen2_seats.txt, etc.

User Functionalities:

Let users:

View available movies and show times

Select a screen and show

View current seat layout (10x10 matrix)

Select seats to book (e.g., A5, B3, F7)



University of Central Punjab

(Incorporated by Ordinance No. XXIV of 2002 promulgated by Government of the Punjab)

FACULTY OF INFORMATION TECHNOLOGY

Enter personal info (Name, CNIC, Phone)

Confirm booking and receive ticket ID

Cancel a ticket using ticket ID

For booking:

Update seat status from 'A' to 'B'

Calculate and display ticket cost

Save booking info to bookings.txt

For cancellation:

Change seat from 'B' back to 'A'

Record cancellation in cancelled.txt

Deduct refund amount (e.g., 90% returned)

Use **pointers** to handle user data and pass seat info by reference.

File Handling Requirements:

movies.txt → Movie and screen details

bookings.txt → Booking records (name, CNIC, ticket ID, screen, seats, total)

cancelled.txt → Cancelled ticket records

screenX_seats.txt → Seat matrix per screen

revenue.txt → Daily earnings summary

Each file should be opened in the appropriate mode (append/write/read), and data should persist between sessions.

Functions to Implement:

Write the following core functions using **pointers and file handling**:

addMovie()

displayMovies()

bookTickets()

cancelBooking()

printTicket()

generateDailyReport()

saveSeatLayout()

loadSeatLayout()

Use 2D arrays in char seats[10][10] for seat availability tracking.

Validation Rules:

No duplicate ticket IDs

Cannot book already booked or blocked seats

CNIC must be 13 digits

Maximum 5 seats can be booked in one transaction

Sample Input:

Select Screen: 2

View Seat Map:

A A B A A A A A A A

A A B A A A A B B A

A A A A A A A A A A

...



University of Central Punjab

(Incorporated by Ordinance No. XXIV of 2002 promulgated by Government of the Punjab)

FACULTY OF INFORMATION TECHNOLOGY

Enter Seats to Book: A1, A2, A4

Enter Name: Saad Ali

CNIC: 3520212345671

Phone: 03011234567

Booking Confirmed

Ticket ID: TCK-2025-084

Cancel Booking:

Enter Ticket ID: TCK-2025-084

Booking Found. Refund: 90%

Booking Cancelled Successfully.

Sample Output:

Movie Added to Screen 2: Avengers – Show Time: 5:30 PM – Price: Rs. 800

Seats Booked Successfully!

Ticket ID: TCK-2025-084

Total Amount: Rs. 2,400

Cancelled: TCK-2025-084

Refund Issued: Rs. 2,160

Total Bookings Today: 74

Total Revenue: Rs. 59,200

@@@@@@@@@@@@@@@@@BEST OF LUCK@@@@@@@@@@@@@@@@@